Prometheus and Grafana

pull, query, alert, draw

Watch the 4-min video

In one line: Prometheus pulls metrics from each service's `/metrics` endpoint, stores them as time series and evaluates PromQL rules; Alertmanager turns firing rules into grouped, routed notifications; Grafana draws dashboards from the same queries.

1 Fifty services, one metrics stack

You need metrics from fifty services, alerts that page the right people, and dashboards the whole team understands. The most common open-source answer is Prometheus to collect and query, and Grafana to show. Prometheus is a CNCF graduated project, and the pair is the default metrics stack in most Kubernetes clusters; the concepts (scrape, PromQL, rules, routing) carry over to managed versions.

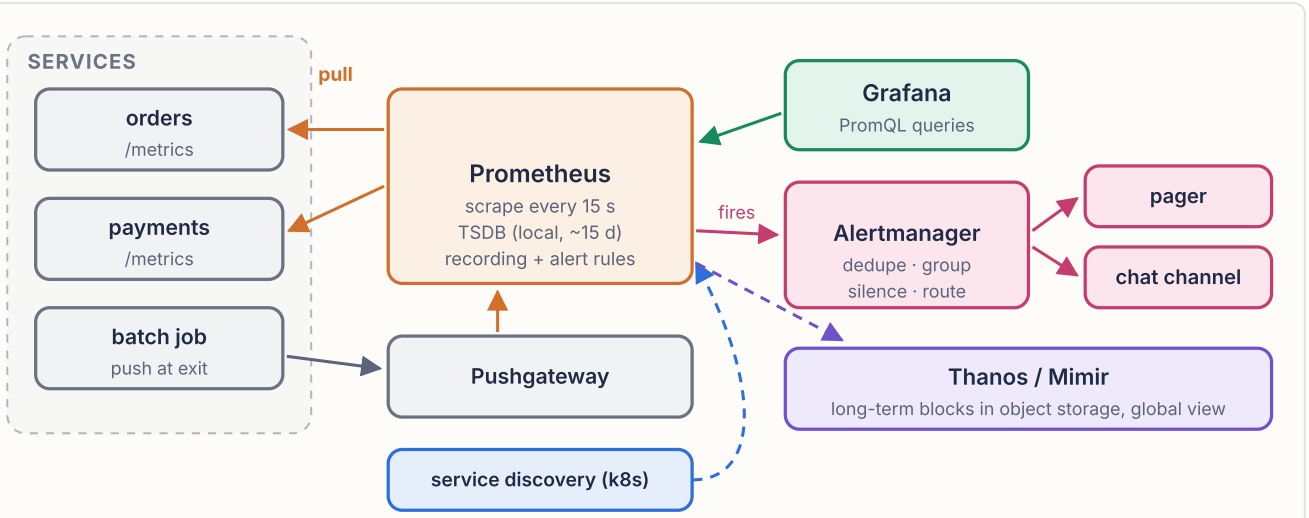


Figure 1. Prometheus scrapes targets found by service discovery, evaluates rules, sends alerts to Alertmanager; Grafana queries it; Thanos or Mimir keep long-term history.

2 Pull-based scraping

Each service exposes a plain-text endpoint, usually `/metrics`, listing its counters, gauges and histograms (lesson 61). On an interval (the config default is 1 minute; **15 s** is the common setting) Prometheus scrapes every target and stores the samples in its local time-series database (default retention 15 days). Targets come from **service discovery**, e.g. the Kubernetes API, so new pods are picked up automatically.

	Pull (Prometheus)	Push (StatsD, OTLP push)
Dead target	visible: `up == 0`	silence looks like "no traffic"
Debugging	open `/metrics` in a browser	inspect what the agent received
Short-lived jobs	need the Pushgateway	natural fit
Behind NAT / firewalls	server must reach target	target only needs outbound

The Pushgateway is only for batch jobs that exit before a scrape: it holds their last values. It is not a general push pipeline.

3 PromQL: rate, sum by, recording rules

A counter is turned into a per-second **rate** over a window: `rate(x[5m])`. Use a window of several scrape intervals (commonly ≥ 4×). `sum by (service)` aggregates away every other label. Expensive queries become **recording rules**: evaluated on a schedule and stored as a new series (`job:http_errors:ratio5m`), so dashboards and alerts stay fast.

```
# error ratio per service, last 5 minutes
sum by (service) (rate(http_requests_total{code=~"5.."}[5m]))
  / sum by (service) (rate(http_requests_total[5m]))

# p99 latency per route (the Grafana panel in the example)
histogram_quantile(0.99,
  sum by (le, route) (rate(http_request_duration_seconds_bucket[5m])))

# alerting rule
- alert: HighErrorRatio
  expr: job:http_errors:ratio5m > 0.02      # recording rule, precomputed
  for: 10m
  labels: {severity: page}
  annotations: {runbook_url: https://wiki/runbooks/high-errors}
```

Figure 2. The three queries you will write most: error ratio, p99 from a histogram, and an alerting rule with a duration and a runbook.

4 Alerting rules and Alertmanager

An alerting rule is a PromQL condition plus a duration: `for: 10m` means the condition must hold for 10 minutes before it fires, which filters blips. Prometheus sends firing alerts to **Alertmanager**, which **deduplicates** (two Prometheus replicas, one notification), **groups** related alerts (`group_by: [service]`, defaults `group_wait 30s`, `repeat_interval 4h`), applies **silences** during maintenance and **inhibitions** (suppress "high latency" when "cluster down" fires), and **routes** by label: `severity=page` to the pager, warnings to a chat channel (lesson 67).

5 Grafana and long-term storage

Grafana queries Prometheus and draws time series, heatmaps (great for histograms) and single stats. **Variables** such as `$service` and `$region` let one dashboard serve every service. Keep dashboards as JSON **in Git**, next to the service. One Prometheus server is a single node with limited local history, so for a global view across clusters and months of data, teams add **Thanos** or **Grafana Mimir**, which store blocks in object storage (S3, GCS, Azure Blob) and query across them.

WORKED EXAMPLE: A P99 LATENCY PANEL

The service exposes `http_request_duration_seconds` as a histogram. The panel query (Figure 2): take `rate()` of the `_bucket` series over 5 minutes, `sum by (le, route)` (you must keep `le`, the bucket boundary, or the quantile can't be computed), then `histogram_quantile(0.99, …)`. Result: one p99 line per route. A red threshold at **500 ms** shows which routes break the target; the same expression, as a recording rule, feeds the alert.

COMMON MISTAKES

- `sum by (route)` without `le` before `histogram_quantile`: the result is meaningless.
- `rate()` over a window shorter than two scrapes, giving gaps or empty results.
- Using the Pushgateway for long-running services: you lose `up` and keep stale values forever.
- Alerts with no `for:` duration, so every 15-second blip pages someone.

6 Key terms

Scrape: Prometheus pulling /metrics from a target

PromQL: Prometheus query language

Alertmanager: dedupes, groups, silences and routes alerts

Service discovery: finding targets automatically (e.g. Kubernetes)

Recording rule: precomputed query stored as a new series

Thanos / Mimir: long-term, global Prometheus storage

7 Interview questions

Q1. How does Prometheus collect metrics? BASIC

It pulls them: it scrapes each target's /metrics endpoint on an interval (often 15 s), with targets found through service discovery; short-lived jobs use the Pushgateway.

Q2. Write a PromQL query for error rate. BASIC

```
sum(rate(http_requests_total{code=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))
```

Q3. Two Prometheus replicas scrape the same targets for HA. Will on-call get two pages? SCENARIO

No: both send to Alertmanager, which deduplicates identical alerts (and an Alertmanager cluster gossips so it notifies once).

Q4. Dashboards with 30 days of p99 across 50 services load slowly. Fix? SCENARIO

Precompute the per-service histogram aggregations with recording rules, and serve long ranges from Thanos or Mimir with downsampling.

Q5. Why is pull useful for detecting failures? TRICKY

Prometheus knows which targets should exist, so a target that stops answering produces `up == 0`; with push, silence looks like no traffic.

Q6. Why can't you just average per-pod p99 in Grafana? TRICKY

Quantiles don't average. Sum the histogram buckets across pods by `le`, then apply histogram_quantile once.

Next: SD-N-005 · Distributed Tracing backendroadmap.com